

Task 1: CPU Fundamentals

1. What is a CPU?

CPU (Central Processing Unit) is the main processing component of a computer. It executes instructions from programs and performs calculations and logical operations.

The CPU is often called the "**brain**" of the computer, although modern computers also rely heavily on GPUs, NPUs, and other specialized processors.

A CPU performs four basic activities:

1. **Fetch:** Gets an instruction from memory.
2. **Decode:** Determines what the instruction means.
3. **Execute:** Performs the required operation.
4. **Store/Write Back:** Saves the result.

This is called the **Fetch-Decode-Execute Cycle**.

2. CPU Cores

A **core** is an individual processing unit inside a CPU.

A CPU can have multiple cores:

- 2 cores → Dual-core
- 4 cores → Quad-core
- 6 cores → Hexa-core
- 8 cores → Octa-core
- 16+ cores → Common in high-performance CPUs

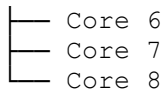
Each core can execute instructions independently.

Example

An 8-core CPU can potentially work on several tasks simultaneously:

CPU

- Core 1
- Core 2
- Core 3
- Core 4
- Core 5



More cores are especially useful for:

- Video rendering
- 3D rendering
- Compiling software
- Virtual machines
- Multitasking
- Scientific computing

3. Threads

A **thread** is a sequence of instructions that can be scheduled for execution.

Modern CPUs may allow one physical core to handle more than one hardware thread.

For example:

```
8 Physical Cores
    ↓
16 Hardware Threads
```

This does **not** mean the CPU has 16 physical cores.

Intel commonly calls its simultaneous multithreading technology **Hyper-Threading**, while AMD commonly uses **SMT (Simultaneous Multithreading)**.

4. Clock Speed

Clock speed represents how quickly a CPU's clock cycles occur.

It is normally measured in:

- MHz
- GHz

For example:

4.0 GHz = 4 billion clock cycles per second.

However, higher GHz does **not automatically mean a faster CPU**.

Performance also depends on:

- CPU architecture
- Instructions per cycle
- Number of cores
- Cache
- Memory speed
- Workload
- Power limits

So comparing CPUs using GHz alone can be misleading.

5. CPU Cache

Cache is very fast memory located inside or close to the CPU.

It stores frequently needed data and instructions so that the CPU does not always have to access slower RAM.

Typical cache levels:

L1 Cache

- Smallest
- Fastest
- Usually dedicated to individual cores

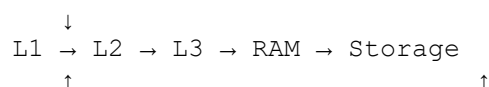
L2 Cache

- Larger than L1
- Slightly slower

L3 Cache

- Larger than L2
- Usually shared between multiple cores
- Slower than L1/L2 but much faster than RAM

Fastest



Small

Large

A CPU with a large cache can have an advantage in workloads where frequently accessed data fits inside the cache.

6. Hyper-Threading

Intel Hyper-Threading is Intel's implementation of simultaneous multithreading.

It allows one physical CPU core to expose multiple logical processors to the operating system.

For example:

```
4 Physical Cores
    ↓
8 Logical Threads
```

The operating system sees 8 logical processors.

Hyper-Threading/SMT improves utilization of CPU resources, especially when one thread is waiting for something.

However:

2 threads on one core are NOT equivalent to 2 physical cores.

7. How Does the CPU Execute a Program?

Suppose we have:

```
x = 10 + 20
```

The general process is:

```
Program
    ↓
Compiler / Interpreter
    ↓
Machine Instructions
    ↓
RAM
    ↓
CPU Fetches Instructions
    ↓
```

Decode
↓
Execute
↓
Result

Fetch

The CPU uses the **Program Counter (PC)** to know where the next instruction is located.

Decode

The CPU's instruction-decoding circuitry determines what operation is required.

Execute

The appropriate CPU units perform the operation.

For example:

10 + 20
↓
Arithmetic Logic Unit
↓
30

The result may then be stored in a register, cache, or memory.

Task 2: Process States and Process Life Cycle

1. What is a Process?

A **process** is a program that is currently being executed.

For example:

Program on disk:
Chrome.exe

↓
Running Chrome

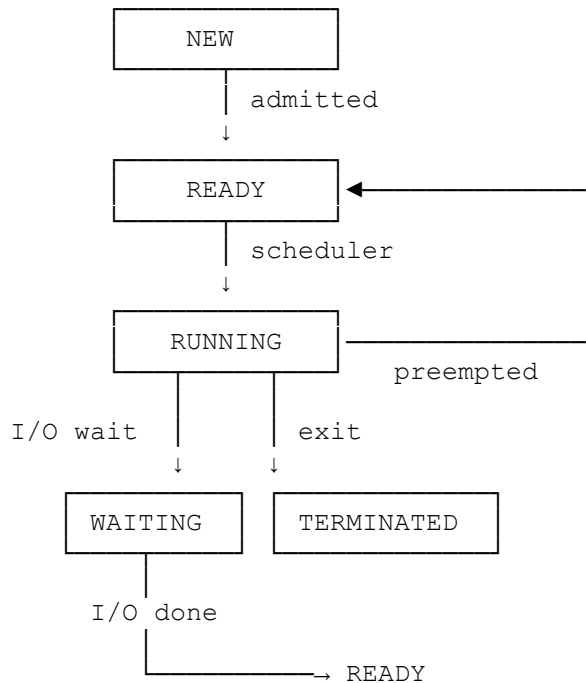
↓
Process

A process contains things such as:

- Program instructions
 - Variables
 - Memory
 - CPU registers
 - Program counter
 - Open files
 - Operating-system resources
-

2. Process Life Cycle

A process normally moves through several states:



3. New State

The process has just been created.

Example:

You open Microsoft Word.

User launches Word
↓
Process is created
↓
NEW

The operating system prepares the necessary resources.

4. Ready State

The process is ready to execute but is waiting for CPU time.

Example:

Process A → Ready
Process B → Ready
Process C → Ready

↓
CPU Scheduler chooses one

Being **Ready** does **NOT** mean the process is currently executing.

It means:

"I'm ready. Give me the CPU!"

5. Running State

The process is currently executing instructions on the CPU.

READY
↓
CPU Scheduler
↓
RUNNING

Only processes/threads assigned CPU execution are running.

6. Waiting / Blocked State

A process enters the waiting state when it cannot continue until some event occurs.

Common reasons:

- Waiting for disk I/O
- Waiting for keyboard input
- Waiting for network data
- Waiting for another process
- Waiting for a resource

Example:

```
Program requests file
      ↓
Disk operation
      ↓
WAITING
      ↓
File arrives
      ↓
READY
```

The CPU can execute another process while the first process waits.

7. Terminated State

The process has finished execution or has been stopped.

```
RUNNING
      ↓
Program finishes
      ↓
TERMINATED
```

The operating system can then release the process's resources.

8. Important Process Transitions

Transition	Meaning
New → Ready	Process has been admitted by OS
Ready → Running	CPU scheduler selects process
Running → Ready	Process is preempted
Running → Waiting	Process waits for I/O/event
Waiting → Ready	Event/I/O completes

Transition	Meaning
Running → Terminated	Process finishes

Task 3: Processes vs Threads

1. Process Definition

A **process** is an independent executing program with its own address space and resources.

Example:

```

Chrome Process
  ↓
Own memory space
Own resources

```

2. Thread Definition

A **thread** is a smaller execution unit inside a process.

One process can contain multiple threads.

```

Process
├── Thread 1
├── Thread 2
├── Thread 3
└── Thread 4

```

Threads within the same process share many resources.

3. Process vs Thread

Feature	Process	Thread
Memory	Separate address space	Shares process memory
Communication	More expensive	Easier/faster
Creation	Relatively expensive	Relatively cheap
Context switching	More expensive	Usually cheaper
Isolation	High	Lower
Failure	One process can fail independently	Thread failure can affect process

Feature	Process	Thread
Resource usage	Higher	Lower
Best for	Independent applications	Parallel tasks within an application

4. Memory

Processes

Each process normally has its own virtual address space.

Process A → Memory A

Process B → Memory B

Threads

Threads within a process share:

- Code
- Heap
- Global variables
- Many process resources

But each thread generally has its own:

- Stack
- Registers
- Program counter

```

Process
├── Shared memory
├── Thread 1 → own stack
├── Thread 2 → own stack
└── Thread 3 → own stack

```

5. Communication

Processes need mechanisms such as:

- Pipes
- Sockets
- Shared memory

- Message queues

This is called **IPC (Inter-Process Communication)**.

Threads can communicate more easily because they share memory.

However, shared memory creates synchronization problems such as:

- Race conditions
 - Deadlocks
 - Data corruption
-

6. Context Switching

Context switching occurs when the CPU stops executing one task and starts executing another.

For processes, the OS may need to switch:

- Registers
- Program counter
- Memory mappings
- Scheduling information

Thread switching can be cheaper when switching between threads belonging to the same process because much of the address space is shared.

7. When Should You Use Processes?

Processes are useful when you need:

- Strong isolation
- Independent applications
- Better fault isolation
- Separate memory spaces

Example:

```
Browser Process
+
Music Player Process
+
Text Editor Process
```

If the browser crashes, the other processes can continue.

8. When Should You Use Threads?

Threads are useful when several tasks belong to the same application.

Example:

A video editor could have:

```
Main Thread
├── UI Thread
├── Rendering Thread
├── Audio Thread
└── Background Processing Thread
```

Task 4: CPU Upgrade Guide

Before upgrading a CPU, you need to determine whether the computer actually supports a processor upgrade.

This is particularly important for **laptops**, because laptop CPUs are often soldered directly to the motherboard.

1. Check the CPU Socket

Desktop CPUs use a specific socket.

Examples include:

- Intel LGA sockets
- AMD AM4
- AMD AM5

The new CPU must physically and electrically match the motherboard's supported socket.

```
CPU Socket
↓
Does CPU fit?
↓
YES → Continue checking
```

NO → Cannot use CPU

But matching the socket alone is **not enough**.

2. Check Chipset Compatibility

The motherboard chipset determines which CPUs and features are supported.

For example:

```
CPU
↓
Socket compatibility
↓
Chipset compatibility
↓
BIOS support
```

Two CPUs may use the same physical socket but still not both be supported by a particular motherboard.

3. Check BIOS/UEFI Compatibility

A motherboard may require a specific BIOS/UEFI version to recognize a newer CPU.

Therefore:

1. Find motherboard model.
2. Visit manufacturer support page.
3. Check supported CPU list.
4. Check minimum BIOS version.
5. Update BIOS if necessary.

Important: BIOS updates should be performed carefully because an interrupted update can cause serious problems.

4. Check TDP / Power Requirements

The motherboard's power-delivery system and cooling system must be capable of handling the new processor.

TDP is related to thermal and power design, but it should **not** be treated as a universal measurement of the exact electricity a CPU will consume.

Check:

- CPU power requirements
 - Motherboard VRM capability
 - Cooler capability
 - PSU capacity
 - Manufacturer recommendations
-

5. Check Cooling

A more powerful CPU may produce more heat.

You may need:

- Better CPU cooler
 - Larger heatsink
 - Better airflow
 - More capable laptop cooling system
-

6. Manufacturer Support

The safest source is the motherboard or laptop manufacturer's official CPU compatibility list.

For laptops, also check:

- Exact laptop model
- CPU soldering
- BIOS support
- Motherboard design
- Cooling system

Desktop

CPU upgrades are often possible.

Laptop

CPU upgrades are frequently impossible because the CPU is soldered to the motherboard.

Task 5: CPU Manufacturers and Processor Comparison

1. Major CPU Manufacturers

Intel

Intel produces CPUs for:

- Desktop PCs
- Laptops
- Workstations
- Servers

Important families include:

- Core
- Core Ultra
- Xeon

AMD

AMD produces:

- Ryzen
- Threadripper
- EPYC

AMD is particularly strong in:

- High-core-count CPUs
- Gaming
- Workstations
- Servers

Apple Silicon

Apple's M-series uses ARM-based architecture.

Examples:

M1
M2
M3
M4

Higher-end variants include:

Pro
Max
Ultra

Apple Silicon is designed around **system-on-chip (SoC)** integration, combining CPU, GPU, memory-related components, and specialized accelerators.

Qualcomm Snapdragon X

Qualcomm's Snapdragon X platform targets Windows PCs.

Major variants include:

- Snapdragon X
- Snapdragon X Plus
- Snapdragon X Elite

They use ARM-based CPU designs and emphasize:

- Power efficiency
 - Battery life
 - Integrated AI/NPU processing
-

2. x86 vs x86-64 vs ARM

x86

x86 is a CPU instruction-set architecture family originating from Intel's 8086.

It traditionally refers to 32-bit versions of the architecture, although "x86" is also sometimes used informally for the broader family.

x86-64

x86-64 extends x86 to support 64-bit processing.

It is also known as:

- AMD64
- Intel 64

It is widely used in:

- Desktop PCs
 - Gaming computers
 - Servers
 - Workstations
-

ARM

ARM uses a different instruction-set architecture family.

ARM processors are widely known for:

- Power efficiency
- Mobile devices
- Embedded systems
- Modern laptops

Examples:

Apple M-series
Qualcomm Snapdragon X

Basic comparison

Feature	x86/x86-64	ARM
Major PC use	Very common	Rapidly growing
Power efficiency	Varies	Traditionally strong
Compatibility	Huge PC software ecosystem	Growing rapidly
Examples	Intel, AMD	Apple, Qualcomm
Common devices	PCs, servers	Phones, tablets, laptops, embedded

3. Desktop vs Laptop vs Workstation vs Server CPUs

Type	Main Goal	Typical Characteristics
Desktop	General/high performance	High performance, upgradeable
Laptop	Performance + battery	Power efficient
Workstation	Professional workloads	Many cores, high memory capacity
Server	Continuous multi-user workloads	High core count, reliability, large memory

Desktop

Designed for:

- Gaming
- Programming
- General productivity

Laptop

Prioritizes:

- Battery life
- Thermal efficiency
- Low power consumption

Workstation

Designed for:

- CAD
- 3D rendering
- Scientific computing
- Professional video production

Server

Designed for:

- Databases
- Cloud services
- Virtual machines

- Web servers
 - Enterprise applications
-

4. Intel Naming Conventions

Intel consumer processor families traditionally include:

Core i3
Core i5
Core i7
Core i9

Intel has also introduced **Core Ultra** branding for newer processor generations.

General positioning

i3 → Entry level
i5 → Mainstream
i7 → High performance
i9 → Enthusiast/high performance

Actual performance depends heavily on the specific generation and model.

Intel Xeon

Xeon is aimed primarily at professional/workstation/server applications.

Typical features can include:

- High core counts
 - Large memory support
 - Enterprise features
 - ECC support on supported platforms
-

Intel Suffixes

K

Unlocked desktop processor.

Core i7-14700K

Can support CPU overclocking on compatible platforms.

KF

Unlocked processor without integrated graphics.

F

No integrated graphics.

H

High-performance laptop processor.

HX

Higher-power/high-performance laptop processor class.

U

Designed around lower power consumption for mobile systems.

P

Laptop processors designed for performance-oriented thin-and-light systems.

T

Lower-power desktop processor class.

5. AMD Naming Conventions

Main Ryzen families:

Ryzen 3
Ryzen 5
Ryzen 7
Ryzen 9

Generally:

Ryzen 3 → Entry
Ryzen 5 → Mainstream
Ryzen 7 → High performance
Ryzen 9 → Enthusiast/high performance

AMD Threadripper

Designed for extremely demanding workstation workloads.

Useful for:

- 3D rendering
 - Simulation
 - Professional content creation
 - Heavy multitasking
-

AMD EPYC

AMD EPYC is designed for servers and data centers.

It focuses on:

- High core counts
 - Large memory capacity
 - Enterprise workloads
-

AMD Suffixes

X

Performance-oriented desktop processor.

G

Historically used on Ryzen processors with integrated Radeon graphics.

X3D

Uses AMD's 3D V-Cache technology.

This can provide significant benefits in some gaming workloads.

HX

High-performance mobile processor.

HS

High-performance, efficiency-oriented mobile processor class.

U

Low-power mobile processor class.

6. Apple M-Series

Apple's M-series includes:

M1
M2
M3
M4

Within a generation:

M4
├── M4
├── M4 Pro
├── M4 Max
└── M4 Ultra

Generally, moving upward in the family provides greater:

- CPU capability
- GPU capability
- Memory bandwidth/capacity
- Professional workload performance

The exact specifications depend on the specific model.

7. Qualcomm Snapdragon X

Qualcomm's PC-oriented Snapdragon X family includes:

Snapdragon X
Snapdragon X Plus
Snapdragon X Elite

The platform uses ARM-based CPUs and includes an NPU designed for AI workloads.

8. How to Read a Processor Model

Example:

Intel Core i7-14700K

Breakdown:

Intel	→ Manufacturer
Core i7	→ Product tier
14	→ Generation indicator
700	→ Model/tier number
K	→ Unlocked/high-performance suffix

Another example:

AMD Ryzen 7 7800X3D	
AMD	→ Manufacturer
Ryzen 7	→ Product tier
7000	→ Generation/family
800	→ Model
X3D	→ 3D V-Cache

Important: naming schemes change between generations, so model numbers should always be interpreted using the manufacturer's documentation for that generation.

9. CPU Specifications

Cores

More cores generally help with highly parallel workloads.

Examples:

- Rendering
 - Compilation
 - Encoding
 - Virtual machines
-

Threads

More hardware threads can improve multitasking and parallel workloads.

However:

Threads are not equivalent to physical cores.

Clock Speed

Higher clock speeds can help performance, particularly in workloads that cannot use many cores.

But architecture and IPC also matter.

Cache

Larger cache can reduce the need to access slower memory.

Different types:

L1
L2
L3

TDP / Power Efficiency

TDP is related to thermal design and power behavior.

Power efficiency asks:

How much useful work can the processor perform for the power it consumes?

This is particularly important in:

- Laptops
 - Battery-powered devices
 - Data centers
-

10. iGPU vs dGPU

iGPU

Integrated GPU is built into or integrated with the processor/platform.

Advantages:

- Lower cost
- Lower power consumption
- No separate graphics card required

Good for:

- Web browsing
 - Office applications
 - Basic graphics
 - Some light gaming
 - Video playback
-

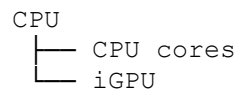
dGPU

Dedicated GPU is a separate graphics processor.

Examples include NVIDIA and AMD graphics cards.

Advantages:

- Much higher graphics performance
- Better gaming
- Better 3D rendering
- Better GPU-accelerated AI workloads



OR

CPU + Separate dGPU

11. AI / NPU

Modern processors increasingly include an **NPU (Neural Processing Unit)**.

An NPU is specialized for AI and machine-learning workloads.

Examples:

- Apple Neural Engine
- Qualcomm Hexagon NPU
- Intel NPU
- AMD XDNA-based AI accelerators

NPUs can perform certain AI workloads efficiently while consuming less power than using general-purpose CPU cores.

However:

Having an NPU does not automatically make a processor the best choice for AI/ML.

For large machine-learning models, a powerful discrete GPU with sufficient VRAM can be much more important.

12. Best Processor Type for Different Tasks

There is no single "best CPU" for everything.

Task	What to prioritize
Programming	Strong single-core + multiple cores
Gaming	Strong CPU performance + GPU
AI/ML	CPU + powerful GPU + RAM/VRAM
Video Editing	Many cores + strong GPU + memory bandwidth
Software Development	Cores + RAM + fast storage
General Productivity	Efficiency + responsive performance

Programming

A modern **Core i5/i7, Core Ultra 5/7, or Ryzen 5/7** is usually more than enough.

For large projects and virtualization, more cores and RAM become valuable.

Gaming

Look for:

- Strong single-thread performance
- Good gaming architecture
- Sufficient cache
- Strong dGPU

AMD's **X3D** processors are particularly notable for gaming-oriented systems.

AI & Machine Learning

For serious local AI:

CPU
+
Large RAM
+
Powerful NVIDIA/AMD GPU
+
Large VRAM

The GPU is often the star of the show.

Video Editing

Prefer:

- Multiple CPU cores
- Strong GPU
- Hardware video encoders/decoders
- Plenty of RAM
- Fast storage

Software Development

Prioritize:

- 6+ modern cores when budget allows
- Good single-core performance
- 16–32 GB RAM or more depending on workload
- Fast SSD

General Productivity

You don't need an extreme CPU.

A modern efficient mid-range processor is usually ideal.

Task 6: Big O Notation

1. What is Big O?

Big O notation describes how the resource requirements of an algorithm grow as the input size increases.

It is mainly used to describe:

- **Time complexity**
 - **Space complexity**
-

2. Time Complexity

Time complexity describes how the amount of work performed by an algorithm grows as input size n increases.

Example:

```
for i in range(n):  
    print(i)
```

If n doubles, approximately twice as many iterations occur.

Therefore:

$O(n)$

3. Space Complexity

Space complexity describes how much additional memory an algorithm needs as input size increases.

Example:

```
new_list = []  
  
for item in data:  
    new_list.append(item)
```

If `data` contains `n` elements, the new list also grows with `n`.

Therefore:

$O(n)$ space

4. $O(1)$ Constant Time

The operation takes approximately the same amount of work regardless of input size.

Example:

```
x = numbers[0]
```

Array/list indexing is typically **$O(1)$** .

```
n = 10          → ~1 operation  
n = 1,000       → ~1 operation  
n = 1,000,000 → ~1 operation
```

5. $O(\log n)$ Logarithmic

The number of operations grows slowly as `n` increases.

Classic example:

Binary Search

Suppose there are 1,000,000 sorted elements.

Binary search repeatedly divides the search space approximately in half.

1,000,000

↓
500,000
↓
250,000
↓
...

Complexity:

$O(\log n)$

6. $O(n)$ Linear

Work grows directly with input size.

Example:

```
for item in numbers:  
    print(item)
```

If:

```
n = 10 → 10 operations  
n = 100 → 100 operations  
n = 1000 → 1000 operations
```

Therefore:

$O(n)$

7. $O(n \log n)$

Common in efficient sorting algorithms.

Examples:

- Merge Sort
- Heap Sort
- Average-case Quick Sort

Example:

Sorting n elements
↓
Divide into smaller pieces
↓
Process pieces
↓
Combine

Complexity:

$O(n \log n)$

8. $O(n^2)$ Quadratic

Usually occurs with nested loops.

Example:

```
for i in range(n):  
    for j in range(n):  
        print(i, j)
```

For:

```
n = 10 → 100 operations  
n = 100 → 10,000 operations  
n = 1000 → 1,000,000 operations
```

Complexity:

$O(n^2)$

9. $O(2^n)$ Exponential

The amount of work approximately doubles with each additional input element.

A common example is a naive recursive solution to Fibonacci:

```
def fibonacci(n):  
    if n <= 1:  
        return n  
    return fibonacci(n-1) + fibonacci(n-2)
```

Its naive recursive implementation has approximately exponential time complexity.

$O(2^n)$ is generally considered very expensive for large n .

10. $O(n!)$ Factorial

Factorial complexity grows extremely quickly.

A classic example is generating every possible ordering of n objects.

For example, with:

```
3 objects → 3! = 6 arrangements
5 objects → 5! = 120
10 objects → 10! = 3,628,800
```

Complexity:

$O(n!)$

This becomes impractical very quickly.

11. Complexity Comparison

From generally best to worst growth:

```
O(1)
↓
O(log n)
↓
O(n)
↓
O(n log n)
↓
O(n2)
↓
O(2n)
↓
O(n!)
```

Visual idea:

Growth

